

The Emulator

"Introspection is all you need."

Covenants on Bitcoin without softfork on Arkade

@bonomat

Freedom Tech Summit Prague · June 2026

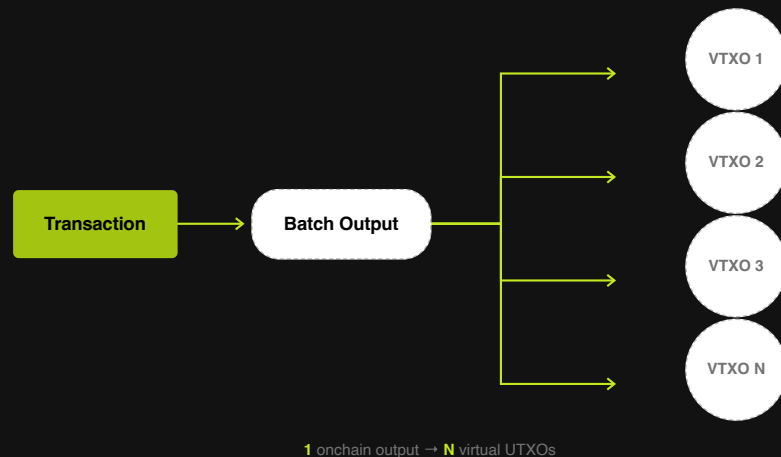
CHAPTER 01 / 05

Arkade

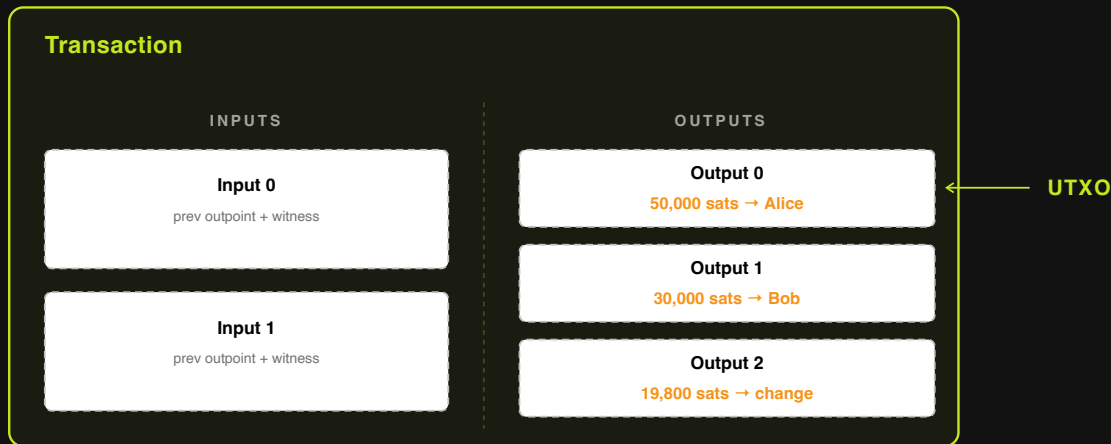
Not a Bitcoin L2

What is Arkade?

- **Not A Bitcoin L2** Pre-signed bitcoin transactions anchored to a single on-chain output - for fast, cheap, off-chain payments.
- **VTXOs** = *virtual* UTXOs that live inside that shared batch output (batched into one onchain tx).
- Every VTXO is a **2-of-2** between user and operator + 1 one unilateral exit path.
- Trade-off: instant + non-custodial, but **every move needs the operator to co-sign.**



Anatomy of a Bitcoin transaction



Each output has a **value** and a **scriptPubKey**. Each input references a previous output's outpoint.

A **VTXO** is one of these outputs — with a script that says *what's allowed to spend it*.

Anatomy of a VTXO

```
vtxoScript = Taproot(  
  internalKey: UNSPENDABLE,  
  scriptPaths :  
    [  
      // Collaborative path (default)  
      checkSig(userPK) && checkSig(operatorPK),  
  
      // Unilateral exit path  
      checkSig(userPK) && relativeTimelock(exitDelay)  
    ]  
)
```

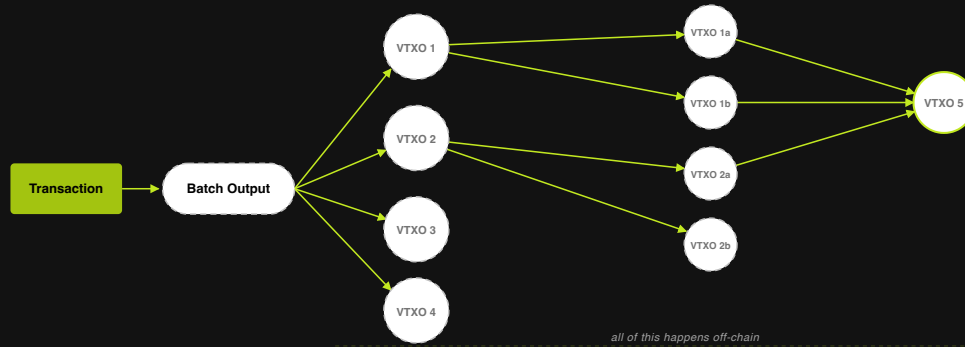
Collaborative path

User **and** operator co-sign every move. Instant settlement, no on-chain footprint.

Unilateral exit

After `exitDelay`, the user alone can claim the VTXO on-chain. The operator **cannot** steal funds.

Spends compose, off-chain



One on-chain anchor. Many off-chain spends. The DAG grows without touching the chain.

What can you build on Arkade?

Instant Bitcoin payments

Lightning-grade speed: no per-user channels to open, no liquidity management.

Other asset and stablecoins

Issued assets inside VTXOs.

Trading & atomic swaps

Atomic swaps between Arkade assets, or cross-chain to Lightning / onchain.

Arkade Script

Extra opcodes on top of Bitcoin Script: introspection, hashing, arithmetic, **covenants**.

CHAPTER 02 / 05

Covenants

The constraint Bitcoin Script can't express (yet)

What is a covenant?

- Bitcoin Script today gates **who** can spend a UTXO.
- A covenant gates **how** it can be spent.

Regular UTXO

"anyone with key x can move this *anywhere*"

Covenant UTXO

"this can only be spent to output y , for amount z , after time T "

A spending constraint that depends on the *transaction itself*, not just signatures and preimages.

Why you'd want them

- **Lightning async receive** — pay-on-preimage without both parties online at swap time.
- **Non-interactive atomic swaps** — trade assets without online requirement.
- **Vaults** — funds can only move to your cold address, or anywhere else after a delay.
- **Games & fun scripts** — provably-fair betting settled entirely by covenant (e.g. dice, coin flips).
- **Derivatives** — advanced financial products such as options, futures, etc.

The common thread: each one wants the script to **see the spending transaction** — its outputs, its amounts, its shape.

What Bitcoin Script can — and can't — do

Today, Script constrains:

- *who* signs — `OP_CHECKSIG` , multisig, MuSig2
- *when* it spends — `OP_CLTV` , `OP_CSV`
- *what secret* is revealed — hashlocks

Script cannot:

- read the spending tx's outputs
- enforce "this output must look like *that*"
- check amounts, scriptPubKeys, or versions
- chain commitments across transactions

Bitcoin scripts today

The soft-fork road 🐢

OP_CTV · OP_CAT · OP_CSFS · OP_VAULT ·
SIGHASH_ANYPREVOUT

Each one is years of debate, safety review, and politics. We've been "**almost there**" for a while now.

Meanwhile, the pragmatic path 🐇

Run a signer that **only co-signs if your script says it's OK.**

No fork required.

That signer is the **Emulator** →

CHAPTER 03 / 05

The Emulator

A signer that only co-signs if your script says it's OK

New Opcodes

Arkade-script gives the covenant the introspection Bitcoin Script lacks — the script can read the spending tx:

`OP_INSPECTOUTPUTVALUE` : an output's amount

`OP_INSPECTINPUTVALUE` : another input's amount

`OP_INSPECTOUTPUTSCRIPTPUBKEY` : its destination

`OP_INSPECTPACKET` : attached data (args, reveals)

`OP_CAT` · `OP_MUL` · `OP_DIV`

...and more

None of these exist on-chain 🚫

No soft-fork has shipped them --> Bitcoin can't validate a VTXO locked by them. So how do we use them *today*?

The script-committed pubkey

The covenant leaf is an ordinary multisig — except the Emulator's key is **tweaked by the script**:

```
P_committed = P_emulator + H_tag("ArkScriptHash", arkadeScript) · G
```

└ the daemon's └ a BIP-340 tagged hash of the exact
long-term key script bytes, taken as a scalar

The pubkey *is* the commitment

Standard BIP-341 taproot tweaking, with a project tag.
Anyone can recompute `P_committed` from the script —
swap the script and the key changes, so the leaf is bound to
one exact script.

Only a passing run unlocks it

The Emulator *can* derive the signing key for `P_committed`
(it holds `P_emulator`'s secret + the script). It only does so
after running that script and seeing it pass.

The taproot "internal key + script commitment" trick — repurposed so a *signer* enforces the script, instead of consensus.

Where does the script live?

Not in the input's tapscript. The committed script rides the **spending transaction itself** — as a packet in an **OP_RETURN** extension output.

Spending tx

INPUTS

Input 0 · **covenant VTXO**
Emulator-tweaked leaf

Input 1 · other VTXO

OUTPUTS

Output 0 · **pot** → **winner**

Output 1 · OP_RETURN
extension · 0 sats



EmulatorPacket · one per covenant input

```
vin      : 0
script   : <arkade-script bytes> ← the covenant
witness  : [output_idx, other_input_idx]
```

The Emulator reads the packet, re-derives `H_tag("ArkScriptHash", script)` to confirm it matches the tweaked key, runs the script against *this* tx — then signs. Apps can ride **extra** packets too — e.g. CoinFlip's secret reveals, pulled inside the script with `INSPECTPACKET`.

A real OP_RETURN, byte for byte

Banco swap fulfillment — the simplest case: **one** covenant packet, empty witness, no reveals. The whole blob fits in a direct push.

```
scriptPubKey of the fulfillment tx's extension output (vout · 0 sats)
```

```
6a          OP_RETURN
38          push · 56 bytes      (≤ 75 → no PUSHDATA needed)
41 52 4b    "ARK"      Arkade extension magic

01          packet: EmulatorPacket (type 0x01)
33          payload = 51 bytes
01          1 entry
00 00       vin = 0              u16-LE
2e          script = 46 bytes
00 cf ...   the fulfill covenant (0 INSPECTOUTPUTVALUE ...)
00          witness = 0 bytes     (covenant reads output 0 directly)
```

"Pay the maker ≥ wantAmount ." One packet, one entry — the Emulator runs it and co-signs. Next: CoinFlip rides extra packets too.

VTXO, with an Emulator leaf

```
vtxoScript = Taproot(  
  internalKey: UNSPENDABLE,  
  scriptPaths: [  
    // Covenant path – the script does the work  
    checkSig(operatorPK) && checkSig(P_committed),  
    //                                     ↑ tweaked by H_tag("ArkScriptHash", arkadeScript)  
  
    // Unilateral exit – the safety valve  
    checkSig(userPK) && relativeTimelock(exitDelay)  
  ]  
)
```

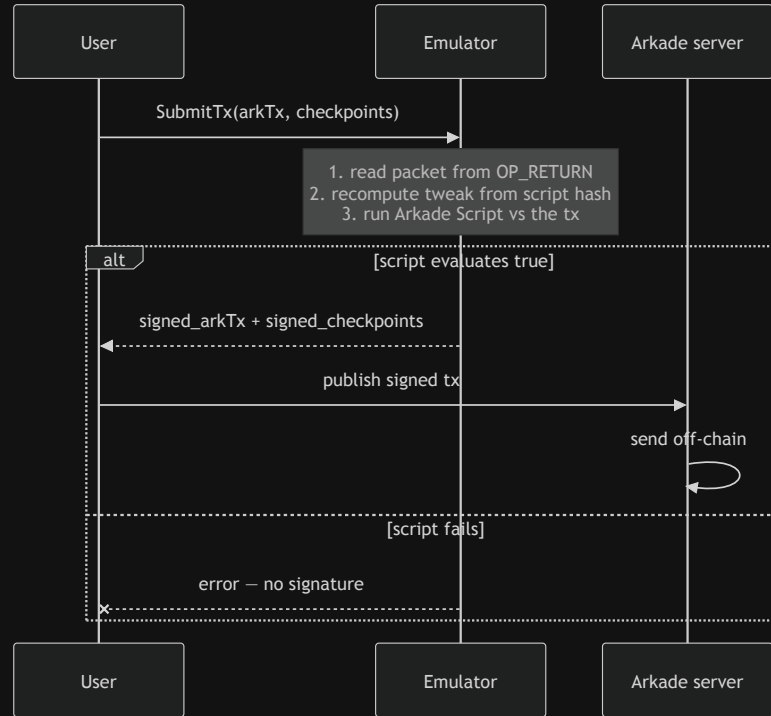
Covenant path

Operator + Emulator. The Emulator signs **only** if the committed Arkade script evaluates true against the spending tx. **No user signature required** which makes it covenant-like.

Unilateral exit

User alone, after `exitDelay`. Operator and Emulator can both vanish; the user still walks away on-chain.

How the Emulator signs



CHAPTER 04 / 05

Show me the code

Real-world covenants, today

Banco: non-interactive swaps

`github.com/arkade-os/banco`

Maker creates an offer, locks funds in a covenant VTXO, then **goes offline**. **Taker** discovers the offer and fulfills it unilaterally.

Why the maker can disappear

The covenant script enforces, on the fulfillment tx:

- output 0 pays the **maker's** scriptPubKey
- output 0 value $\geq$ wantAmount (sats or assets)

No maker signature at fulfillment.

A swap, concretely

Alice holds **\$100**, wants **sats**:

1. Locks the **\$100** in a covenant VTXO
2. Covenant: *"spendable only if $\geq X$ sats pay Alice"*
3. Goes **offline** — anyone may fulfill
4. Taker takes the \$100, pays Alice $\geq X$ sats — one atomic tx

`src/maker.ts` · `src/taker.ts` · `src/offer.ts`

Banco, in arkade-script

Maker — the covenant it commits `offer.ts`

```
# fulfillScript — wanting BTC, full fill
# pins the taker's fulfillment tx:

0  INSPECTOUTPUTVALUE           # output 0 value
<wantAmount> GREATERTHANOREQUAL # ... ≥ wantAmount
VERIFY
0  INSPECTOUTPUTSCRIPTPUBKEY     # output 0 spk
1  EQUALVERIFY                   # ... taproot (v1)
<makerWP> EQUAL                  # ... pays the MAKER
```

"Whoever spends me **must** send ≥ `wantAmount` to the maker." No maker signature needed.

Taker — reconstruct leaf & hand off `taker.ts`

```
// the fulfill leaf: 2-of-2, no taker key
fulfillLeaf = Multisig[
  serverPubKey,
  computeArkadeScriptPublicKey(
    emulatorPubkey, // ← tweaked by
    covenantScript, // the covenant
  ),
]

// 1 build tx: output 0 → maker, ≥ wantAmount
// 2 attach covenantScript as a packet
// 3 submit to the Emulator
```

The Emulator runs the covenant against the tx, signs the tweaked slot; **arkd** finalizes.

Asset swaps swap line 1 for `<wantTxid> 0 INSPECTOUTASSETLOOKUP VERIFY` ; partial fills add `MUL/DIV` ratio math + a self-send change leg.

CoinFlip - provably fair *and* provably paid

`github.com/arkade-os/coinflip`

A "PROVABLY-FAIR" GAME

Proves the **roll** is honest — and nothing else.

- House can **refuse to pay** a winner
- ...or **stall** indefinitely
- ...or **vanish** with the pot

COINFLIP'S COVENANT

Both player **and house** escrow into **one covenant** up front.

- Commit-reveal keeps the roll **unbiasable**
- Funds from **both** parties committed
- Winner claims **unilaterally**

How a game is created

Each party funds **their own** escrow VTXO --> same covenant rules. Settlement spends **both** in one transaction.

player stake VTXO

player funds it · `playerStake`

house stake VTXO

house funds it · `houseStake`

same covenant taptree · two separate VTXOs



Settlement tx · 2 in → 1 out

INPUTS

Input 0 · player stake

Input 1 · house stake

OUTPUT

Output 0 · **whole pot → winner**

The house server assembles & submits this tx. The win leaf is a 2-of-2 [`arkd` & `Emulator`] : the Emulator co-signs only if the roll + payout check out (once per input), then `arkd` finalizes. **Neither player nor house signs the payout** — so whoever builds it still can't cheat.

CoinFlip covenants script

```
# Player-win leaf – the Emulator runs this before it co-signs
# entry stack: [output_idx, other_input_idx] (covenant args)

# 1 · pull both reveals (they ride the spending tx as packets)
0x10 INSPECTPACKET VERIFY      # player reveal
0x11 INSPECTPACKET VERIFY      # creator reveal

# 2 · bind each reveal to its commitment
DUP SHA256 <playerHash> EQUALVERIFY
SWAP DUP SHA256 <creatorHash> EQUALVERIFY

# 3 · the coin: roll = (creatorBit + playerBit) mod 2
1 LEFT BIN2NUM   SWAP   1 LEFT BIN2NUM # first byte of each reveal → its bit (0/1)
ADD  2 MOD      0 1 WITHIN              # player wins iff roll = 0 (the bits match)
```

Bits match → player: $1 + 1 = 2 \rightarrow 2 \bmod 2 = 0 \rightarrow 0 \in [0,1) \checkmark$

player wins

Bits differ → house: $0 + 1 = 1 \rightarrow 1 \bmod 2 = 1 \rightarrow 1 \notin [0,1) \times$

house wins

Same leaf on **both** escrows · house-win leaf = same body + `OP_NOT` .

② CAN'T STALL THE PAYOUT

...then pin the payout

```
# the predicate left `playerWins` on the stack — require it:
VERIFY                                     # predicate must hold — then...

# atomic-sweep covenant: pin destination + amount
INSPECTINPUTVALUE  <otherStake> EQUALVERIFY
DUP  INSPECTOUTPUTSCRIPTPUBKEY 1 EQUALVERIFY <winnerScript> EQUALVERIFY
INSPECTOUTPUTVALUE <pot> EQUAL
```

Full leaf = predicate · VERIFY · atomicSweep . The Emulator derives its tweaked key — and signs — only when the whole thing evaluates true.

CHAPTER 05 / 05

Summary

Cosigning is all you need

The trick

Tweak a signer's pubkey with `H_tag("ArkScriptHash", script)`. The pubkey is a commitment to the script.

The result

Covenant-like behaviour for Arkade VTXOs. **Today**, no soft-fork needed.

Questions?